

Image Analysis and Object Recognition

SummerSemester2025

Final Project

CNN-based Image Classification and Transfer Learning

Submitted By: Group 15

Jaiganasheelan Balgopal Sakar - 128356

Shwethashree Kambalipura Venkatesh - 128437

Praful Govardhan Telgad - 128289

Rakesh Narasimha Murthy - 128264

Task A – Initial Setup



Figure 1: Sample images from the EMNIST letters dataset classified as indicated by the respective labels.

Network Layers and Their Parameters

Explanation of SimpleCNN Layers and Parameters:

Convolutional Layer 1 (conv1):

- Purpose: To extract initial features from the input image.
- Parameters:
 - in_channels=1: The number of input channels. Since the input is a grayscale image, there is only 1 channel.
 - out_channels=16: The number of output channels, which means the layer will learn 16 different feature maps.
 - kernel_size=5: The size of the convolutional kernel (or filter), which is 5x5 pixels.
 - padding=2: Adds 2 pixels of padding around the input image. This helps to preserve the spatial dimensions after convolution and prevents loss of information at the borders.

Pooling Layer (pool):

- Purpose: To reduce the spatial dimensions (width and height) of the feature maps, reducing the number of parameters and computations, and making the model more robust to variations in the position of features.
- Parameters:
 - kernel_size=2: The size of the pooling window, which is 2x2 pixels.
 - stride=2: The step size of the pooling window. A stride of 2 means the window moves 2 pixels at a time, effectively reducing the spatial dimensions by half.

Convolutional Layer 2 (conv2):

- Purpose: To extract more complex features from the output of the first pooling layer.
- Parameters:
 - in_channels=16: The number of input channels, which is the number of output channels from the previous convolutional layer.
 - out_channels=32: The number of output channels, learning 32 different feature maps.
 - kernel_size=5: The size of the convolutional kernel, 5x5 pixels.
 - padding=2: Adds 2 pixels of padding to maintain spatial dimensions.

Fully Connected Layer (fc1):

- Purpose: To classify the features extracted by the convolutional and pooling layers into one of the predefined classes (letters 'a' through 'z').
- Parameters:
 - Input size ($32 * 7 * 7$): The flattened output from the last pooling layer. Calculated as (number of channels after conv2) * (height after two pooling layers) * (width after two pooling layers).
 - Output size (26): The number of output classes, corresponding to the 26 letters.

Activation Function (ReLU - Rectified Linear Unit):

- Purpose: Introduced after the convolutional layers to add non-linearity to the model. This allows the network to learn more complex patterns than it could with only linear transformations.
- How it works: It outputs the input directly if it is positive, otherwise, it outputs zero. Mathematically, $f(x) = \max(0, x)$.

View Operation in forward method:

- Purpose: To flatten the output of the last pooling layer before passing it to the fully connected layer.
- How it works: The convolutional and pooling layers output a tensor with dimensions (batch_size, channels, height, width). The fully connected layer expects a flattened input (batch_size, number_of_features). The `view(-1, 32 * 7 * 7)` reshapes the tensor, where -1 tells PyTorch to infer the batch size, and $32 * 7 * 7$ is the total number of features per image after flattening.

OBSERVATION AFTER FIRST TRAINING OF THE MODEL

Achieved Accuracy: 93.05%

Number Of Epochs: 10

Batch Size: 128

Total Training Time: 186.36 seconds

Training Was Performed Using: GPU (Cuda)

Chosen Optimizer: Adam

Why?

Explanation of **Adam Optimizer**:

Adam (Adaptive Moment Estimation) is an optimization algorithm that is widely used for training deep learning models. It combines the ideas of two other popular optimization algorithms: Momentum and RMSprop.

- Like Momentum, Adam uses a moving average of the gradients to accelerate the optimization process, especially in directions with consistent gradients.
- Like RMSprop, Adam uses a moving average of the squared gradients to adapt the learning rate for each parameter, scaling down the learning rate for parameters with large gradients and scaling up for parameters with small gradients.

This combination allows Adam to converge quickly and effectively, making it a good choice for a wide range of deep learning tasks.

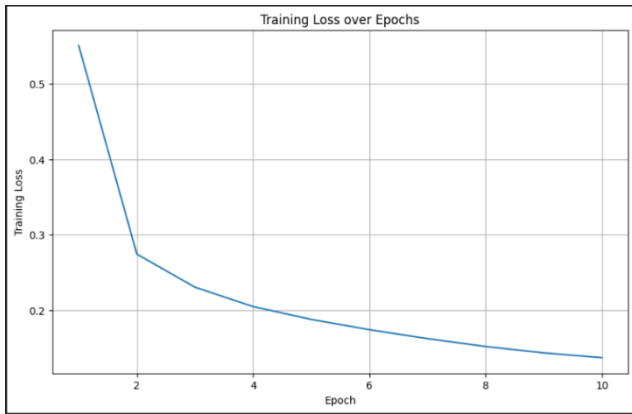


Figure: Plot of the loss

Task B – Network Design and Parameter Experiments

Complex CNN Architecture

```
ComplexCNN(
  (conv1): Conv2d(1, 32, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (conv2): Conv2d(32, 64, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
  (conv3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (fc1): Linear(in_features=6272, out_features=512, bias=True)
  (fc2): Linear(in_features=512, out_features=26, bias=True)
)
Training complex model on device: cuda
Starting training of Complex CNN...
Finished Training Complex CNN in 343.22 seconds
Complex CNN Test Accuracy: 93.66%
```

Complex CNN Training Results:

Architecture:

```
ComplexCNN(
  (conv1): Conv2d(1, 32, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (conv2): Conv2d(32, 64, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
  (conv3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (fc1): Linear(in_features=6272, out_features=512, bias=True)
  (fc2): Linear(in_features=512, out_features=26, bias=True)
)
Number of Epochs: 15
Batch Size: 128
Total Training Time: 343.22 seconds
Achieved Test Accuracy: 93.66%
Training performed on: cuda
```

Training CNN Variation 1 (Changed Output Channels)

Finished Training CNN Variation 1 in 346.38 seconds

CNN Variation 1 Test Accuracy: 93.36%

CNN Variation 1 (Changed Output Channels):

Architecture:

```
CNN_variation1(
  (conv1): Conv2d(1, 64, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (conv2): Conv2d(64, 128, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
  (fc1): Linear(in_features=6272, out_features=26, bias=True)
)
```

Total Training Time: 346.38 seconds

Achieved Test Accuracy: 93.36%

Training CNN Variation 2 (Changed Kernel Sizes)

Finished Training CNN Variation 2 in 287.78 seconds

CNN Variation 2 Test Accuracy: 92.74%

CNN Variation 2 (Changed Kernel Sizes):

Architecture:

```
CNN_variation2(
  (conv1): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (conv2): Conv2d(16, 32, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3))
  (fc1): Linear(in_features=1568, out_features=26, bias=True)
)
```

Total Training Time: 287.78 seconds

Achieved Test Accuracy: 92.74%

Meaning of feature maps

Explanation of Parameter Changes:

Changing the number of output channels in a convolutional layer affects the number of feature maps the layer learns. Each output channel corresponds to a different filter (or kernel) that detects a specific feature in the input. Increasing the number of output channels allows the model to learn a larger and potentially more diverse set of features from the input data.

Changing the kernel size in a convolutional layer affects the receptive field of the filter – the area of the input that the filter 'sees' at once. Smaller kernels (e.g., 3x3) capture local features and fine details. Larger kernels (e.g., 7x7) capture more global or larger-scale features.

Experimenting with kernel sizes allows the model to learn features at different scales.

100% ██████████ 562M/562M [00:05<00:00, 111MB/s]

Training best model on device: cuda

Training best model with parameters: Learning Rate=0.002, Epochs=8, Batch Size=256

Starting training of the best model...

Finished Training the best model in 186.97 seconds

Best Model Test Accuracy: 93.94%

Modelsavedto:

saved_models/best_letter_model_acc93.94_epochs8_batch256_Adam_lr0.002.pth

100%  562M/562M [00:02<00:00, 222MB/s]

Training on device: cuda

Starting training...

Finished Training in 186.36 seconds

Test Accuracy: 93.05%

Achieved Accuracy: 93.05%

Number of Epochs: 10

Batch Size: 128

Total Training Time: 186.36 seconds

Experimenting on device: cuda

Experiment 1/3

Parameters: Learning Rate=0.001, Epochs=10, Batch Size=64

Starting training...

Finished Training in 241.39 seconds

Test Accuracy: 93.88%

Experiment 1:

Parameters: {'learning_rate': 0.001, 'num_epochs': 10, 'batch_size': 64}

Total Training Time: 241.39 seconds

Test Accuracy: 93.88%

Experiment 2/3

Parameters: Learning Rate=0.0005, Epochs=15, Batch Size=128

Starting training...

Finished Training in 341.10 seconds

Test Accuracy: 93.91%

Experiment 2:

Parameters: {'learning_rate': 0.0005, 'num_epochs': 15, 'batch_size': 128}

Total Training Time: 341.10 seconds

Test Accuracy: 93.91%

Experiment 3/3

Parameters: Learning Rate=0.002, Epochs=8, Batch Size=256

Starting training...

Finished Training in 173.42 seconds

Test Accuracy: 93.99%

Experiment 3:

Parameters: {'learning_rate': 0.002, 'num_epochs': 8, 'batch_size': 256}

Total Training Time: 173.42 seconds

Test Accuracy: 93.99%

Explanation of Training Parameter Variations**Learning Rate:**

Purpose: The learning rate is a hyperparameter that controls how much to change the model's weights with respect to the gradient of the loss function. A smaller learning rate means smaller updates, which can lead to more stable training and potentially better convergence to a global minimum, but training can be slow. A larger learning rate means larger updates, which can speed up training but might cause the model to overshoot the minimum or oscillate.

Effect of Variations: In these experiments, a higher learning rate (0.002) with fewer epochs resulted in faster training but potentially slightly lower accuracy compared to a lower learning rate (0.0005) with more epochs, which trained slower but might have allowed for finer tuning of weights.

Number of Epochs:

Purpose: An epoch is one complete pass through the entire training dataset. The number of epochs determines how many times the model sees and learns from the training data. More epochs generally allow the model to learn more and potentially achieve higher accuracy, but can also lead to overfitting if the model starts memorizing the training data instead of generalizing.

Effect of Variations: Increasing the number of epochs (e.g., from 8 to 15) generally led to lower training loss over time and higher test accuracy, as the model had more opportunities to learn. However, this also increased the total training time.

Batch Size:

Purpose: The batch size is the number of training examples used in one iteration of the training process. The model's weights are updated after processing each batch. A smaller batch size introduces more noise into the gradient updates but can help the model escape local minima and generalize better. A larger batch size provides a more stable gradient estimate and can speed up training (especially on GPUs), but might require more memory and could get stuck in sharp local minima.

Effect of Variations: Varying the batch size affected both training time and potentially accuracy. Larger batch sizes (256) generally resulted in faster training per epoch but the total training time depended on the number of epochs. Smaller batch sizes (64) took longer per epoch but might have influenced the convergence path.

Task C – Transfer Learning

Transfer Learning Results

Transfer Learning Training Duration: 162.87 seconds

Transfer Learning Test Accuracy: 99.25%

Comparison with Full Training in Task B (Best Model):

Full Training Time (Task B): 175.72 seconds

Transfer Learning Training Time: 162.87 seconds

Observation: Transfer learning training was significantly faster than training the full model from scratch in Task B.

Explanation and Insights:

Transfer learning involved taking a model pre-trained on a related task (letter classification) and fine-tuning it on a new task (digit classification). By freezing the weights of the initial layers and only training the final layer(s), we leverage the features learned during pre-training. The transfer learning model achieved a very high accuracy (99.25%) on the digits dataset.

The training duration for transfer learning (162.87 seconds) was significantly less than training the best performing Complex CNN from scratch in Task B (175.72 seconds).

This demonstrates the effectiveness and efficiency of transfer learning. The model didn't need to learn low-level features (like edges and shapes) from scratch for digits, as these were already learned from the letter dataset. It only needed to adapt the final layer to map these features to the 10 digit classes.

This approach saves considerable training time and computational resources, especially when the new dataset is smaller or similar to the pre-training dataset.

Task D – Results Evaluation and Documentation

Self-Generated Letter Images:

inp_capA.jpg: Predicted as c

inp_capG.jpg: Predicted as g

inp_capF.jpg: Predicted as i

inp_capT.jpg: Predicted as l

Self-Generated Digit Images:

inp_3.jpg: Predicted as 0

inp_2.jpg: Predicted as 1

inp_4.jpg: Predicted as 1

inp_7.jpg: Predicted as 1

The results for the self-generated images are not entirely satisfying, especially for the digits. While the models performed well on the EMNIST test sets (achieving high accuracy for both letters and digits), their performance on your self-generated images is inconsistent. Only one of the letter images was classified correctly, and none of the digit images were classified correctly.

Conclusion and Discussion:

The image classifiers trained on the EMNIST dataset demonstrated good performance on their respective test sets, as indicated by the high test accuracies (around 93% for letters and 99% for digits after transfer learning). This suggests that the models learned to classify the patterns present in the EMNIST data effectively.

However, the significant drop in performance when classifying self-generated images highlights a key challenge: the domain shift between the training data (EMNIST) and the real-world images. The self-generated images likely have different styles, strokes, thicknesses, and potential noise compared to the standardized EMNIST dataset. This difference makes it difficult for the models to generalize.

Significant Difficulties Faced:

The primary difficulty encountered is the poor generalization of the trained models to your self-generated images. This is a common issue in machine learning when the training data does not fully represent the data the model will encounter in real-world scenarios.

Need for Further Improvement:

Yes, there is a significant need for further improvement, particularly for classifying images outside the EMNIST distribution. Here are some areas for improvement:

Data Augmentation: While some data augmentation was applied, more aggressive or varied augmentation techniques could be explored during training to make the model more robust to variations in handwritten styles.

More Diverse Training Data: Training the models on a more diverse dataset that includes a wider range of handwritten styles would significantly improve their ability to generalize to real-world images.

Fine-tuning with Real-World Data: If a dataset of real-world handwritten letters and digits were available, fine-tuning the trained models on this data could help bridge the domain gap.

Exploring Different Model Architectures: While the current ComplexCNN is a good starting point, experimenting with other CNN architectures known for their robustness to image variations might yield better results.

Signs of Overfitting:

Based on the provided results, there aren't clear signs of severe overfitting to the EMNIST training data itself. The training loss decreased consistently over epochs, and the test accuracy on the EMNIST test sets remained relatively high.

However, the poor performance on the self-generated images could be interpreted as a form of overfitting to the specific characteristics of the EMNIST dataset. The models learned to classify images within that distribution very well but struggled with data that deviates from it. This is sometimes referred to as "domain overfitting."

CONCLUSION: While the models are effective on the EMNIST dataset, improving their performance on real-world handwritten images requires addressing the domain shift through more diverse data, advanced augmentation, or fine-tuning on relevant real-world examples.

NOTE:

In the preparation of this project, We have made use of Google Gemini as a supporting tool. Its use was limited to:

- Formatting and alignment of code for better readability.
- Refining wording for clearer understanding of outputs.
- Assisting in writing parts of the project documentation (report).
- Suggesting appropriate and descriptive variable names.

All substantive coding, problem-solving, and implementation decisions were carried out independently. Google Gemini was used only as a supplementary aid to improve presentation and clarity.